

UNIT - 1 (Introduction in Python)

Problem 1: Basic Variable Assignment and Printing

```
name = "Alice"  
age = 30  
is_student = True  
print(f"Name: {name}")  
print(f"Age: {age} years old")  
print(f"Is Student: {is_student}")
```

OUTPUT:-

```
Name: Alice  
Age: 30 years old  
Is Student: True
```

Problem 2: Basic Arithmetic Operations

```
num1 = 25  
num2 = 7  
addition = num1 + num2  
subtraction = num1 - num2  
multiplication = num1 * num2  
division = num1 / num2
```

```
modulus = num1 % num2  
print(f"Numbers: {num1} and {num2}")  
print(f"Addition: {num1} + {num2} = {addition}")  
print(f"Subtraction: {num1} - {num2} = {subtraction}")  
print(f"Multiplication: {num1} * {num2} =  
{multiplication}")  
print(f"Division: {num1} / {num2} = {division}")  
print(f"Modulus: {num1} % {num2} = {modulus}")
```

OUTPUT:-

```
Numbers: 25 and 7  
Addition: 25 + 7 = 32  
Subtraction: 25 - 7 = 18  
Multiplication: 25 * 7 = 175  
Division: 25 / 7 = 3.5714285714285716  
Modulus: 25 % 7 = 4
```

UNIT-2 (PYTHON PROGRAM FLOWCONTROL CONDITIONAL BLOCKS)

Problem 1: Number Classification (Positive, Negative, Zero)

try:

```
number = float(input("Enter a number: "))
```

```
if number > 0:
```

```
    print(f"The number {number} is positive.")
```

```
elif number < 0:
```

```
    print(f"The number {number} is negative.")
```

```
else:
```

```
    print(f"The number {number} is zero.")
```

```
except ValueError:
```

```
    print("Invalid input. Please enter a valid number.")
```

OUTPUT:-

Enter a number: 10

The number 10.0 is positive.

Problem 2: For Loop with Continue Statement

```
print("Numbers from 1 to 10 (skipping even numbers):")  
for i in range(1, 11):  
    if i % 2 == 0:  
        continue  
    print(i)
```

OUTPUT:

Numbers from 1 to 10 (skipping even numbers):

1
3
5
7
9

UNIT-3 (Python Complex Data Types)

Problem 1: Basic Operations on Strings, Lists, and Tuples

--- 1. String Operations ---

```
print("--- String Operations ---")
```

```
sentence = "Hello Python World" # Hardcoding for non-interactive output
```

```
print(f"Original sentence: '{sentence}'")
```

```
print(f"Length: {len(sentence)}")
```

```
print(f"Uppercase: {sentence.upper()}")
```

```
print(f"Lowercase: {sentence.lower()}")
```

```
print(f"Title case: {sentence.title()}")
```

```
print(f"First 5 characters (slicing): {sentence[:5]}")
```

```
print(f"Last 5 characters (slicing): {sentence[-5:]}")
```

```
search_term = "Python" # Hardcoding for non-interactive output
```

```
if search_term in sentence:
```

```
    print(f"'{search_term}' found at index:  
{sentence.find(search_term)}")
```

```
else:
```

```
    print(f"'{search_term}' not found in the sentence.")
```

```
print("\n" + "="*30 + "\n")
```

--- 2. List Operations ---

```
print("--- List Operations ---")
my_list = [10, 20, 30, 40, 50]
print(f"Original List: {my_list}")
print(f"Sliced list (index 1 to 3): {my_list[1:4]}")
my_list.append(60)
print(f"After appending 60: {my_list}")
my_list.insert(0, 5)
print(f"After inserting 5 at index 0: {my_list}")
if 30 in my_list:
    my_list.remove(30)
    print(f"After removing 30: {my_list}")
else:
    print("30 not found in the list to remove.")
popped_element = my_list.pop(2)
print(f"After popping element at index 2 ({popped_element}):
{my_list}")

print("\n" + "="*30 + "\n")

# --- 3. Tuple Operations ---
print("--- Tuple Operations ---")
my_tuple = ("apple", "banana", "cherry", "date", "elderberry")
print(f"Original Tuple: {my_tuple}")
print(f"Length of tuple: {len(my_tuple)}")
```

```
print(f"Element at index 0: {my_tuple[0]}")  
print(f"Element at index 3: {my_tuple[3]}")  
print(f"Last element: {my_tuple[-1]}")  
print(f"Sliced tuple (index 1 to 3): {my_tuple[1:4]}")  
print(f"Sliced tuple (first three): {my_tuple[:3]}")  
print(f"Sliced tuple (last two): {my_tuple[-2:]}")
```

OUTPUT:-

--- String Operations ---

Original sentence: 'Hello Python World'

Length: 18

Uppercase: HELLO PYTHON WORLD

Lowercase: hello python world

Title case: Hello Python World

First 5 characters (slicing): Hello

Last 5 characters (slicing): World

'Python' found at index: 6

=====

--- List Operations ---

Original List: [10, 20, 30, 40, 50]

Sliced list (index 1 to 3): [20, 30, 40]

After appending 60: [10, 20, 30, 40, 50, 60]

After inserting 5 at index 0: [5, 10, 20, 30, 40, 50, 60]

After removing 30: [5, 10, 20, 40, 50, 60]

After popping element at index 2 (20): [5, 10, 40, 50, 60]

=====

--- Tuple Operations ---

Original Tuple: ('apple', 'banana', 'cherry', 'date', 'elderberry')

Length of tuple: 5

Element at index 0: apple

Element at index 3: date

Last element: elderberry

Sliced tuple (index 1 to 3): ('banana', 'cherry', 'date')

Sliced tuple (first three): ('apple', 'banana', 'cherry')

Sliced tuple (last two): ('date', 'elderberry')

Problem 2: Student Information Management (Dictionaries and Functions)

```
students_data = {}
```

```
def add_student(name, marks):
```

```
    if name in students_data:
```



```
    print(f"Student '{name}' already exists. Use update_marks to  
change marks.")
```

```
else:
```

```
    students_data[name] = marks
```

```
    print(f"Student '{name}' with marks {marks} added successfully.")
```

```
def update_marks(name, new_marks):
```

```
    if name in students_data:
```

```
        students_data[name] = new_marks
```

```
        print(f"Marks for '{name}' updated to {new_marks}.")
```

```
    else:
```

```
        print(f"Student '{name}' not found.")
```

```
def delete_student(name):
```

```
    if name in students_data:
```

```
        del students_data[name]
```

```
        print(f"Student '{name}' deleted successfully.")
```

```
    else:
```

```
        print(f"Student '{name}' not found.")
```

```
def display_all_students():
```

```
    if not students_data:
```

```
        print("No student records to display.")
```

```
    return
```

```
print("\n--- All Student Records ---")  
for name, marks in students_data.items():  
    print(f"Name: {name}, Marks: {marks}")  
print("-----\n")
```

```
def calculate_average_marks():  
    if not students_data:  
        return 0.0  
    total_marks = sum(students_data.values())  
    average = total_marks / len(students_data)  
    return average
```

```
def display_student_details(name):  
    if name in students_data:  
        print(f"\n--- Details for {name} ---")  
        print(f"Name: {name}, Marks: {students_data[name]}")  
        print("-----\n")  
    else:  
        print(f"Student '{name}' not found.")
```

Main execution simulation

```
print("Welcome to Student Management System!")  
add_student("Alice", 85)  
add_student("Bob", 72)
```

```
add_student("Charlie", 90)
add_student("Alice", 95)
display_all_students()
update_marks("Bob", 78)
update_marks("David", 65)
display_all_students()
avg_marks = calculate_average_marks()
print(f"Average marks of all students: {avg_marks:.2f}")
display_student_details("Charlie")
display_student_details("Eve")
delete_student("Bob")
delete_student("Frank")
display_all_students()
```

OUTPUT:-

```
Welcome to Student Management System!
Student 'Alice' with marks 85 added successfully.
Student 'Bob' with marks 72 added successfully.
Student 'Charlie' with marks 90 added successfully.
Student 'Alice' already exists. Use update_marks to change marks.
--- All Student Records ---
Name: Alice, Marks: 85
Name: Bob, Marks: 72
Name: Charlie, Marks: 90
```

Marks for 'Bob' updated to 78.

Student 'David' not found.

--- All Student Records ---

Name: Alice, Marks: 85

Name: Bob, Marks: 78

Name: Charlie, Marks: 90

Average marks of all students: 84.33

--- Details for Charlie ---

Name: Charlie, Marks: 90

Student 'Eve' not found.

Student 'Bob' deleted successfully.

Student 'Frank' not found.

--- All Student Records ---

Name: Alice, Marks: 85

Name: Charlie, Marks: 90

UNIT -4 (Python File Operations)

Problem 1: Reading from a Text File

```
file_name = "sample.txt"
```

```
# Create a sample file for demonstration
```

```
with open(file_name, 'w') as f:
```

```
    f.write("Line 1: This is the first line.\n")
```

```
    f.write("Line 2: This is the second line.\n")
```

```
    f.write("Line 3: And this is the third line.\n")
```

```
    f.write("Line 4: Finally, the fourth line.\n")
```

```
print("--- Using read() ---")
```

```
with open(file_name, 'r') as f:
```

```
    content = f.read()
```

```
    print(content)
```

```
print("--- Using readline() ---")
```

```
with open(file_name, 'r') as f:
```

```
    line1 = f.readline()
```

```
    line2 = f.readline()
```

```
    print(f"First line: {line1.strip()}")
```

```
    print(f"Second line: {line2.strip()}")
```

```
print("--- Using readlines() ---")
with open(file_name, 'r') as f:
    lines = f.readlines()
    print("All lines as a list:")
    for i, line in enumerate(lines):
        print(f"Line {i+1}: {line.strip()}")
```

OUTPUT:-

```
--- Using read() ---
Line 1: This is the first line.
Line 2: This is the second line.
Line 3: And this is the third line.
Line 4: Finally, the fourth line.
--- Using readline() ---
First line: Line 1: This is the first line.
Second line: Line 2: This is the second line.
--- Using readlines() ---
All lines as a list:
Line 1: Line 1: This is the first line.
Line 2: Line 2: This is the second line.
Line 3: Line 3: And this is the third line.
Line 4: Line 4: Finally, the fourth line.
```

Problem 2: Writing to a Text File

```
file_name_write = "output.txt"

# Using write() to write individual lines
with open(file_name_write, 'w') as f:
    f.write("This is the first line written with write().\n")
    f.write("This is the second line also with write().\n")
    f.write("A third line for good measure.\n")

lines_to_write = [
    "This line is from writelines() list entry 1.\n",
    "This is list entry 2, also from writelines().\n",
    "And the final line from the writelines() list.\n"
]

# Use 'a' mode to append to the existing file
with open(file_name_write, 'a') as f:
    f.writelines(lines_to_write)

print(f"Content of '{file_name_write}' after writing:")

with open(file_name_write, 'r') as f:
    print(f.read())
```

OUTPUT:-

Content of 'output.txt' after writing:
This is the first line written with write().

This is the second line also with write().
A third line for good measure.
This line is from writelines() list entry 1.
This is list entry 2, also from writelines().
And the final line from the writelines() list.

UNIT-5 (Python packages)

Problem 1: Data Visualization and Analysis with NumPy, Pandas, Matplotlib

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# --- 1. NumPy: Generate some numerical data ---
days = np.arange(1, 11)
np.random.seed(42)
temperatures = 20 + 5 * np.random.randn(len(days)) + days * 0.5
print(f"NumPy Days array: {days}")
print(f"NumPy Temperatures array: {np.round(temperatures, 2)}")

# --- 2. Pandas: Create a DataFrame from NumPy data ---
data = {'Day': days, 'Temperature_C': temperatures}
```



```
df = pd.DataFrame(data)
print("\nPandas DataFrame:")
print(df)

mean_temp = df['Temperature_C'].mean()
max_temp = df['Temperature_C'].max()
min_temp = df['Temperature_C'].min()
print(f"\nMean Temperature: {mean_temp:.2f}°C")
print(f"Max Temperature: {max_temp:.2f}°C")
print(f"Min Temperature: {min_temp:.2f}°C")

df['Temperature_F'] = (df['Temperature_C'] * 9/5) + 32
print("\nDataFrame with Fahrenheit conversion:")
print(df)
```

--- 3. Matplotlib: Visualize the data ---

```
plt.figure(figsize=(10, 6))
plt.plot(df['Day'], df['Temperature_C'], marker='o', linestyle='-',
color='skyblue', label='Temperature (°C)')
plt.title('Daily Temperatures Over 10 Days')
plt.xlabel('Day')
plt.ylabel('Temperature (°C)')
plt.grid(True)
plt.legend()
```

```
plt.xticks(days)
```

```
plt.tight_layout()
```

```
# plt.show() # Uncomment to display the plot when running locally
```

```
print("\nMatplotlib plot generated (requires plt.show() to display).")
```

OUTPUT:-

```
NumPy Days array: [ 1  2  3  4  5  6  7  8  9 10]
```

```
NumPy Temperatures array: [22.48 20.3  23.51 25.13 25.43 28.74 27.64  
28.18 31.83 30.56]
```

```
Pandas DataFrame:
```

	Day	Temperature_C
0	1	22.483571
1	2	20.297491
2	3	23.509706
3	4	25.129202
4	5	25.432657
5	6	28.742407
6	7	27.642730
7	8	28.184311
8	9	31.833634
9	10	30.561950

Mean Temperature: 26.38°C

Max Temperature: 31.83°C

Min Temperature: 20.30°C

DataFrame with Fahrenheit conversion:

	Day	Temperature_C	Temperature_F
0	1	22.483571	72.470428
1	2	20.297491	68.535484
2	3	23.509706	74.317471
4	5	25.432657	77.778782
5	6	28.742407	83.736333
6	7	27.642730	81.756914
7	8	28.184311	82.731760
8	9	31.833634	89.299540
9	10	30.561950	87.011510

Matplotlib plot generated (requires plt.show() to display).

Problem 2: Tkinter Calculator (GUI application, no direct console output beyond initial setup)

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
class SimpleCalculator:
```

```
    def __init__(self, master):
```

```
        self.master = master
```

```
        master.title("Simple Calculator")
```

```
        self.equation = tk.StringVar()
```

```
        self.equation.set("")
```

```
        self.entry_field = tk.Entry(master, textvariable=self.equation,  
                                     font=('Arial', 20), bd=5, relief='sunken',  
                                     justify='right', state='readonly')
```

```
        self.entry_field.grid(row=0, column=0, columnspan=4, ipadx=8,  
                               ipady=8, pady=5)
```

```
        buttons = [
```

```
            '7', '8', '9', '/',
```

```
            '4', '5', '6', '*',
```

```
            '1', '2', '3', '-',
```

```
            '0', '.', '=', '+'
```

```
        ]
```

```
        row_num = 1
```

```
        col_num = 0
```

```
        for button_text in buttons:
```

```
            tk.Button(master, text=button_text, width=5, height=2,
```

```

        font=('Arial', 14),

        command=lambda text=button_text:
self.button_click(text)).grid(row=row_num, column=col_num, padx=5,
pady=5)

        col_num += 1

        if col_num > 3:

            col_num = 0

            row_num += 1


tk.Button(master, text='C', width=22, height=2,

        font=('Arial', 14),

        command=self.clear_display).grid(row=row_num, column=0,
columnspan=4, padx=5, pady=5)


def button_click(self, text):

    current_equation = self.equation.get()

    if text == '=':

        try:

            result = str(eval(current_equation))

            self.equation.set(result)

        except Exception as e:

            messagebox.showerror("Error", f"Invalid Input or Calculation
Error: {e}")

            self.equation.set("")

    else:

```

```
self.equation.set(current_equation + text)
```

```
def clear_display(self):
```

```
    self.equation.set("")
```

```
# Main execution (this will open a GUI window)
```

```
root = tk.Tk()
```

```
calculator = SimpleCalculator(root)
```

```
root.mainloop()
```

```
print("Tkinter Calculator GUI application is generated. Run the Python  
code to see the interactive window.")
```

OUTPUT:-

